Notes:

- HW2 due today
- HW3 posted, due Oct 28
- Project proposals due Oct 21
- Guest Lecture (Oliver Stueker, Acenet) on October 21

Last class: testing

- decorators
- coverage, interior, edge, corner cases
- unit, integration, regression tests
- CI/CD

A common tool for CI testing is GitHub actions.

Let's commit our code from last week to our git repo, and setup a GitHub actions workflow to run tests.

Two steps to setting this up:

1. Make code directory and copy code and tests to that directory
2. Write a github workflow

Copy from another project, look at `.github/workflows/main.yml`

3 parts:

1. What events trigger the CI? Here, pushes to main and PRs
2. Specify testing environment. Run on ubuntu (could also choose windows or macos). Use python 3.13. Add `requirements.txt` to specify what libraries are needed.
3. Our actions:
    - checkout repo
    - run code with pytest

Add and commit everything. Locally, nothing happens, but go to GitHub webpage

Only a small change, now see an icon next to our commit messges. 3 different things might appear:

- brown dot, indicating actions are in process
- green check, meaning all actions ran and passed
- red x, meaning something failed

When CI runs, click on that icon, get summary report, can click on details, and find information about what was successful or not (standard pytest output)

Make something fail. Know

$$\sin(x)\sin(y) = (\cos(x-y) - \cos(x+y))/2$$

.

Edit `sinc2d` to use this, but make a mistake

Important: do this on a branch, to not mess up passing tests on main!

```
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git checkout -b smaclachlan/trig_identity
Switched to a new branch 'smaclachlan/trig_identity'
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git add code/mymath.py
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git commit -m"Using trig identity for sinc2d"
[smaclachlan/trig_identity 7a0f6ff] Using trig identity for sinc2d
 1 file changed, 1 insertion(+), 1 deletion(-)
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git push
fatal: The current branch smaclachlan/trig_identity has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin smaclachlan/trig_identity

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git push --set-upstream origin smaclachlan/trig_identity
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 10 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 392 bytes | 392.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'smaclachlan/trig_identity' on GitHub by visiting:
remote:      https://github.com/MacLachlanTeaching/CMSC6950_Fall2025_example/pull/new/smaclachlan/trig_identity
remote:
To github.com:MacLachlanTeaching/CMSC6950_Fall2025_example.git
 * [new branch]      smaclachlan/trig_identity -> smaclachlan/trig_identity
branch 'smaclachlan/trig_identity' set up to track 'origin/smaclachlan/trig_identity'.
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan%
```

Create the PR, see failing tests.

```
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git add code/mymath.py
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git commit -m"Fix sign"
[smaclachlan/trig_identity 3b74b85] Fix sign
 1 file changed, 1 insertion(+), 1 deletion(-)
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 10 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 354 bytes | 354.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:MacLachlanTeaching/CMSC6950_Fall2025_example.git
   7a0f6ff..3b74b85  smaclachlan/trig_identity -> smaclachlan/trig_identity
```

Back to PR - tests pass. Now can merge

Locally, once we merge, we switch back to main branch and pull:

```
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
[math-scott02:CMSC6950/Fall2025/CMSC6950_Fall2025_example] smaclachlan% git pull
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 917 bytes | 917.00 KiB/s, done.
From github.com:MacLachlanTeaching/CMSC6950_Fall2025_example
   e1dfd0d..9953da5  main       -> origin/main
Updating e1dfd0d..9953da5
Fast-forward
 code/mymath.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Using CI allows us to do CD (Continuous Delivery). Anyone can pull from main at any time and get working code with latest features.

Now that we know *how* to develop software, we're ready to develop some more specific skills in python/ Our goal over the next weeks will be to learn specifics of the Python language, as well as of the functionality of various Python libraries.

A common problem that we encounter as we start to work with more and more complex libraries is their interdependence. Each library has dependencies on other libraries, and it is possible that two libraries have conflicting requirements.

Some of this is cosmetic:

- Pytorch has very rigid coding style dependencies. If you write code to contribute to Pytorch, these can easily conflict with coding style dependencies used by other projects.

Some of this is pragmatic:

- As libraries are developed, their interfaces can change. When this happens, projects that use the library must adapt. While that code is being written and tested, need to be able to work with older version of the library, or CI/CD model is broken. When a major change happens, some dependencies will adapt quickly, and some more slowly.

Python provides an elegant solution for this through "virtual environments". For each project that you work on, we can work in a self-contained environment, using specific version of Python and some external libraries.

Now, if two projects have conflicting requirements, we don't have to be concerned. Do still need to be aware of dependencies within a single project, but that can't be avoided.

Note: Anaconda provides its own version of virtual environments that works similarly to what I will describe, but with different syntax

From command line, create a virtual environment with `python3 -m venv name_of_venv`. This just creates a directory with some links in it. In the `bin` directory are a number of scripts. My environment uses `tcsh` as the default shell, so I can activate venv by running `source CMSC6950_venv/bin/activate.csh`:

```
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% python3 -m venv CMSC6950_venv
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% ls CMSC6950_venv/
bin             include         lib             pyvenv.cfg
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% ls CMSC6950_venv/bin
activate        activate.fish   pip             pip3.13         python3
activate.csh    Activate.ps1    pip3            python          python3.13
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% source CMSC6950_venv/bin/activate.csh
(CMSC6950_venv) [math-scott02:teaching/CMSC6950/Fall2025] smaclachlan%
```

If you're running bash, don't need `.csh` on the command: `source CMSC6950_venv/bin/activate`

Now anything that I run in terminal is now run in that environment, until I run `deactivate`.

Whole point here is that once I'm in a venv, can start installing packages at command line using pip, and those packages are only "visible" when we're running python from within the venv.

In VSCode, select kernel, Python environments, then can select venv version or standard python

Note that within a venv, `python` and `pip` refer to the python used to create the venv, so if `python` and `python3` are two different versions of python on your system and you run

```
python3 -m venv venv_name
source venv_name/bin/activate
```

within that venv, `python` is the same as `python3`, since that's what created the venv

Whole point of venvs is to handle libraries gracefully. So let's install one

```
(CMSC6950_venv) [math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% pip install pandas
Collecting pandas
  Downloading pandas-2.3.3-cp313-cp313-macosx_11_0_arm64.whl.metadata (91 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-2.3.4-cp313-cp313-macosx_14_0_arm64.whl.metadata (62 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Downloading pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Downloading tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading pandas-2.3.3-cp313-cp313-macosx_11_0_arm64.whl (10.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.7/10.7 MB 17.1 MB/s eta 0:00:00
Downloading numpy-2.3.4-cp313-cp313-macosx_14_0_arm64.whl (5.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.1/5.1 MB 20.1 MB/s eta 0:00:00
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading pytz-2025.2-py2.py3-none-any.whl (509 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Installing collected packages: pytz, tzdata, six, numpy, python-dateutil, pandas
Successfully installed numpy-2.3.4 pandas-2.3.3 python-dateutil-2.9.0.post0 pytz-2025.2 six-1.17.0 tzdata-2025.2

[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: pip install --upgrade pip
(CMSC6950_venv) [math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% python
Python 3.13.3 (main, Apr  8 2025, 13:54:08) [Clang 17.0.0 (clang-1700.0.13.3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
Cmd click to launch VS Code Native REPL
>>> import pandas
>>> ^D
(CMSC6950_venv) [math-scott02:teaching/CMSC6950/Fall2025] smaclachlan%
```

Notice:

```
(CMSC6950_venv) [math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% deactivate
[math-scott02:teaching/CMSC6950/Fall2025] smaclachlan% python3
Python 3.13.3 (main, Apr  8 2025, 13:54:08) [Clang 17.0.0 (clang-1700.0.13.3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
Cmd click to launch VS Code Native REPL
>>> import pandas
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    import platform
    ^^^^^^^^^^^^^
ModuleNotFoundError: No module named 'pandas'
>>> ^D
```

We have pandas library installed in our venv, but not in our main python environment

Running in venv in Jupyter notebook, so we can now use the library in code we write here.

In [1]:

```python
import pandas as pd
```

What is pandas?

- set of data analysis tools in python

We will learn how to deal with numerical data in arrays, but Pandas is focused around 2 data types

1. Series: a list of data values
2. DataFrame: a collection of series

The nice thing about DataFrames is that we can easily collect data records of different types, e.g. names (strings), ages (numbers), etc.

An example of data that benefits from this style of organization is the "gapminder" data set on GDP. Can download data in excel format from `gapminder.org`, but more common to use "comma-separated values" (CSV) format for datasets in Python.

Look at `gapminder_all.csv`. Lots of data!

- First row, labels for each column of a spreadsheet
- Then: 1 row per coountry, with continent name, then country name, then data values
- First each number is the GDP per capita in a given year, adjusted to 2007 value in US dollars, then life expectancies

Can read this data using `pd.read_csv`:

In [2]:

```python
data_americas = pd.read_csv('data/gapminder_gdp_americas.csv')
```

In [3]:

```python
print(data_americas)
```

```
   continent              country  gdpPercap_1952  gdpPercap_1957  \
0   Americas            Argentina     5911.315053     6856.856212
1   Americas              Bolivia     2677.326347     2127.686326
2   Americas               Brazil     2108.944355     2487.365989
3   Americas               Canada    11367.161120    12489.950060
4   Americas                Chile     3939.978789     4315.622723
5   Americas             Colombia     2144.115096     2323.805581
6   Americas           Costa Rica     2627.009471     2990.010802
7   Americas                 Cuba     5586.538780     6092.174359
8   Americas   Dominican Republic     1397.717137     1544.402995
9   Americas              Ecuador     3522.110717     3780.546651
10  Americas          El Salvador     3048.302900     3421.523218
11  Americas            Guatemala     2428.237769     2617.155967
12  Americas                Haiti     1840.366939     1726.887882
13  Americas             Honduras     2194.926204     2220.487682
14  Americas              Jamaica     2898.530881     4756.525781
15  Americas               Mexico     3478.125529     4131.546641
16  Americas            Nicaragua     3112.363948     3457.415947
17  Americas               Panama     2480.380334     2961.800905
18  Americas             Paraguay     1952.308701     2046.154706
19  Americas                 Peru     3758.523437     4245.256698
20  Americas          Puerto Rico     3081.959785     3907.156189
21  Americas  Trinidad and Tobago     3023.271928     4100.393400
22  Americas        United States    13990.482080    14847.127120
23  Americas              Uruguay     5716.766744     6150.772969
24  Americas            Venezuela     7689.799761     9802.466526

    gdpPercap_1962  gdpPercap_1967  gdpPercap_1972  gdpPercap_1977  \
0      7133.166023     8052.953021     9443.038526    10079.026740
1      2180.972546     2586.886053     2980.331339     3548.097832
2      3336.585802     3429.864357     4985.711467     6660.118654
3     13462.485550    16076.588030    18970.570860    22090.883060
4      4519.094331     5106.654313     5494.024437     4756.763836
5      2492.351109     2678.729839     3264.660041     3815.807870
6      3460.937025     4161.727834     5118.146939     5926.876967
7      5180.755910     5690.268015     5305.445256     6380.494966
8      1662.137359     1653.723003     2189.874499     2681.988900
9      4086.114078     4579.074215     5280.994710     6679.623260
10     3776.803627     4358.595393     4520.246008     5138.922374
11     2750.364446     3242.531147     4031.408271     4879.992748
12     1796.589032     1452.057666     1654.456946     1874.298931
13     2291.156835     2538.269358     2529.842345     3203.208066
14     5246.107524     6124.703451     7433.889293     6650.195573
15     4581.609385     5754.733883     6809.406690     7674.929108
16     3634.364406     4643.393534     4688.593267     5486.371089
17     3536.540301     4421.009084     5364.249663     5351.912144
18     2148.027146     2299.376311     2523.337977     3248.373311
19     4957.037982     5788.093330     5937.827283     6281.290855
20     5108.344630     6929.277714     9123.041742     9770.524921
21     4997.523971     5621.368472     6619.551419     7899.554209
22    16173.145860    19530.365570    21806.035940    24072.632130
23     5603.357717     5444.619620     5703.408898     6504.339663
24     8422.974165     9541.474188    10505.259660    13143.950950

    gdpPercap_1982  gdpPercap_1987  gdpPercap_1992  gdpPercap_1997  \
0      8997.897412     9139.671389     9308.418710    10967.281950
1      3156.510452     2753.691490     2961.699694     3326.143191
2      7030.835878     7807.095818     6950.283021     7957.980824
3     22898.792140    26626.515030    26342.884260    28954.925890
4      5095.665738     5547.063754     7596.125964    10118.053180
5      4397.575659     4903.219100     5444.648617     6117.361746
6      5262.734751     5629.915318     6160.416317     6677.045314
7      7316.918107     7532.924763     5592.843963     5431.990415
8      2861.092386     2899.842175     3044.214214     3614.101285
9      7213.791267     6481.776993     7103.702595     7429.455877
10     4098.344175     4140.442097     4444.231700     5154.825496
11     4820.494790     4246.485974     4439.450840     4684.313807
12     2011.159549     1823.015995     1456.309517     1341.726931
13     3121.760794     3023.096699     3081.694603     3160.454906
14     6068.051350     6351.237495     7404.923685     7121.924704
15     9611.147541     8688.156003     9472.384295     9767.297530
16     3470.338156     2955.984375     2170.151724     2253.023004
17     7009.601598     7034.779161     6618.743050     7113.692252
18     4258.503604     3998.875695     4196.411078     4247.400261
19     6434.501797     6360.943444     4446.380924     5838.347657
20    10330.989150    12281.341910    14641.587110    16999.433300
21     9119.528607     7388.597823     7370.990932     8792.573126
22    25009.559140    29884.350410    32003.932240    35767.433030
23     6920.223051     7452.398969     8137.004775     9230.240708
24    11152.410110     9883.584648    10733.926310    10165.495180

    gdpPercap_2002  gdpPercap_2007
0      8797.640716    12779.379640
1      3413.262690     3822.137084
2      8131.212843     9065.800825
3     33328.965070    36319.235010
4     10778.783850    13171.638850
5      5755.259962     7006.580419
6      7723.447195     9645.061420
7      6340.646683     8948.102923
8      4563.808154     6025.374752
9      5773.044512     6873.262326
10     5351.568666     5728.353514
11     4858.347495     5186.050003
12     1270.364932     1201.637154
13     3099.728660     3548.330846
14     6994.774861     7320.880262
15    10742.440530    11977.574960
16     2474.548819     2749.320965
17     7356.031934     9809.185636
18     3783.674243     4172.838464
19     5909.020073     7408.905561
20    18855.606180    19328.709010
21    11460.600230    18008.509240
22    39097.099550    42951.653090
23     7727.002004    10611.462990
24     8605.047831    11415.805690
```

Lots of data. First column is useless here (all the same). Can specify which columns to use with `usecols`

In [4]:

```python
data_americas = pd.read_csv('data/gapminder_gdp_americas.csv', usecols=range(1,14))
print(data_americas)
```

```
                country  gdpPercap_1952  gdpPercap_1957  gdpPercap_1962  \
0             Argentina     5911.315053     6856.856212     7133.166023
1               Bolivia     2677.326347     2127.686326     2180.972546
2                Brazil     2108.944355     2487.365989     3336.585802
3                Canada    11367.161120    12489.950060    13462.485550
4                 Chile     3939.978789     4315.622723     4519.094331
5              Colombia     2144.115096     2323.805581     2492.351109
6            Costa Rica     2627.009471     2990.010802     3460.937025
7                  Cuba     5586.538780     6092.174359     5180.755910
8    Dominican Republic     1397.717137     1544.402995     1662.137359
9               Ecuador     3522.110717     3780.546651     4086.114078
10          El Salvador     3048.302900     3421.523218     3776.803627
11            Guatemala     2428.237769     2617.155967     2750.364446
12                Haiti     1840.366939     1726.887882     1796.589032
13             Honduras     2194.926204     2220.487682     2291.156835
14              Jamaica     2898.530881     4756.525781     5246.107524
15               Mexico     3478.125529     4131.546641     4581.609385
16            Nicaragua     3112.363948     3457.415947     3634.364406
17               Panama     2480.380334     2961.800905     3536.540301
18             Paraguay     1952.308701     2046.154706     2148.027146
19                 Peru     3758.523437     4245.256698     4957.037982
20          Puerto Rico     3081.959785     3907.156189     5108.344630
21  Trinidad and Tobago     3023.271928     4100.393400     4997.523971
22        United States    13990.482080    14847.127120    16173.145860
23              Uruguay     5716.766744     6150.772969     5603.357717
24            Venezuela     7689.799761     9802.466526     8422.974165

    gdpPercap_1967  gdpPercap_1972  gdpPercap_1977  gdpPercap_1982  \
0      8052.953021     9443.038526    10079.026740     8997.897412
1      2586.886053     2980.331339     3548.097832     3156.510452
2      3429.864357     4985.711467     6660.118654     7030.835878
3     16076.588030    18970.570860    22090.883060    22898.792140
4      5106.654313     5494.024437     4756.763836     5095.665738
5      2678.729839     3264.660041     3815.807870     4397.575659
6      4161.727834     5118.146939     5926.876967     5262.734751
7      5690.268015     5305.445256     6380.494966     7316.918107
8      1653.723003     2189.874499     2681.988900     2861.092386
9      4579.074215     5280.994710     6679.623260     7213.791267
10     4358.595393     4520.246008     5138.922374     4098.344175
11     3242.531147     4031.408271     4879.992748     4820.494790
12     1452.057666     1654.456946     1874.298931     2011.159549
13     2538.269358     2529.842345     3203.208066     3121.760794
14     6124.703451     7433.889293     6650.195573     6068.051350
15     5754.733883     6809.406690     7674.929108     9611.147541
16     4643.393534     4688.593267     5486.371089     3470.338156
17     4421.009084     5364.249663     5351.912144     7009.601598
18     2299.376311     2523.337977     3248.373311     4258.503604
19     5788.093330     5937.827283     6281.290855     6434.501797
20     6929.277714     9123.041742     9770.524921    10330.989150
21     5621.368472     6619.551419     7899.554209     9119.528607
22    19530.365570    21806.035940    24072.632130    25009.559140
23     5444.619620     5703.408898     6504.339663     6920.223051
24     9541.474188    10505.259660    13143.950950    11152.410110

    gdpPercap_1987  gdpPercap_1992  gdpPercap_1997  gdpPercap_2002  \
0      9139.671389     9308.418710    10967.281950     8797.640716
1      2753.691490     2961.699694     3326.143191     3413.262690
2      7807.095818     6950.283021     7957.980824     8131.212843
3     26626.515030    26342.884260    28954.925890    33328.965070
4      5547.063754     7596.125964    10118.053180    10778.783850
5      4903.219100     5444.648617     6117.361746     5755.259962
6      5629.915318     6160.416317     6677.045314     7723.447195
7      7532.924763     5592.843963     5431.990415     6340.646683
8      2899.842175     3044.214214     3614.101285     4563.808154
9      6481.776993     7103.702595     7429.455877     5773.044512
10     4140.442097     4444.231700     5154.825496     5351.568666
11     4246.485974     4439.450840     4684.313807     4858.347495
12     1823.015995     1456.309517     1341.726931     1270.364932
13     3023.096699     3081.694603     3160.454906     3099.728660
14     6351.237495     7404.923685     7121.924704     6994.774861
15     8688.156003     9472.384295     9767.297530    10742.440530
16     2955.984375     2170.151724     2253.023004     2474.548819
17     7034.779161     6618.743050     7113.692252     7356.031934
18     3998.875695     4196.411078     4247.400261     3783.674243
19     6360.943444     4446.380924     5838.347657     5909.020073
20    12281.341910    14641.587110    16999.433300    18855.606180
21     7388.597823     7370.990932     8792.573126    11460.600230
22    29884.350410    32003.932240    35767.433030    39097.099550
23     7452.398969     8137.004775     9230.240708     7727.002004
24     9883.584648    10733.926310    10165.495180     8605.047831

    gdpPercap_2007
0     12779.379640
1      3822.137084
2      9065.800825
3     36319.235010
4     13171.638850
5      7006.580419
6      9645.061420
7      8948.102923
8      6025.374752
9      6873.262326
10     5728.353514
11     5186.050003
12     1201.637154
13     3548.330846
14     7320.880262
15    11977.574960
16     2749.320965
17     9809.185636
18     4172.838464
19     7408.905561
20    19328.709010
21    18008.509240
22    42951.653090
23    10611.462990
24    11415.805690
```

Note that first column is really just an index, rather than data, so set a column to use in its place

In [5]:

```python
data_americas = pd.read_csv('data/gapminder_gdp_americas.csv', usecols=range(1,14), index_col='country')
print(data_americas)
```

```
                     gdpPercap_1952  gdpPercap_1957  gdpPercap_1962  \
country
Argentina               5911.315053     6856.856212     7133.166023
Bolivia                 2677.326347     2127.686326     2180.972546
Brazil                  2108.944355     2487.365989     3336.585802
Canada                 11367.161120    12489.950060    13462.485550
Chile                   3939.978789     4315.622723     4519.094331
Colombia                2144.115096     2323.805581     2492.351109
Costa Rica              2627.009471     2990.010802     3460.937025
Cuba                    5586.538780     6092.174359     5180.755910
Dominican Republic      1397.717137     1544.402995     1662.137359
Ecuador                 3522.110717     3780.546651     4086.114078
El Salvador             3048.302900     3421.523218     3776.803627
Guatemala               2428.237769     2617.155967     2750.364446
Haiti                   1840.366939     1726.887882     1796.589032
Honduras                2194.926204     2220.487682     2291.156835
Jamaica                 2898.530881     4756.525781     5246.107524
Mexico                  3478.125529     4131.546641     4581.609385
Nicaragua               3112.363948     3457.415947     3634.364406
Panama                  2480.380334     2961.800905     3536.540301
Paraguay                1952.308701     2046.154706     2148.027146
Peru                    3758.523437     4245.256698     4957.037982
Puerto Rico             3081.959785     3907.156189     5108.344630
Trinidad and Tobago     3023.271928     4100.393400     4997.523971
United States          13990.482080    14847.127120    16173.145860
Uruguay                 5716.766744     6150.772969     5603.357717
Venezuela               7689.799761     9802.466526     8422.974165

                     gdpPercap_1967  gdpPercap_1972  gdpPercap_1977  \
country
Argentina               8052.953021     9443.038526    10079.026740
Bolivia                 2586.886053     2980.331339     3548.097832
Brazil                  3429.864357     4985.711467     6660.118654
Canada                 16076.588030    18970.570860    22090.883060
Chile                   5106.654313     5494.024437     4756.763836
Colombia                2678.729839     3264.660041     3815.807870
Costa Rica              4161.727834     5118.146939     5926.876967
Cuba                    5690.268015     5305.445256     6380.494966
Dominican Republic      1653.723003     2189.874499     2681.988900
Ecuador                 4579.074215     5280.994710     6679.623260
El Salvador             4358.595393     4520.246008     5138.922374
Guatemala               3242.531147     4031.408271     4879.992748
Haiti                   1452.057666     1654.456946     1874.298931
Honduras                2538.269358     2529.842345     3203.208066
Jamaica                 6124.703451     7433.889293     6650.195573
Mexico                  5754.733883     6809.406690     7674.929108
Nicaragua               4643.393534     4688.593267     5486.371089
Panama                  4421.009084     5364.249663     5351.912144
Paraguay                2299.376311     2523.337977     3248.373311
Peru                    5788.093330     5937.827283     6281.290855
Puerto Rico             6929.277714     9123.041742     9770.524921
Trinidad and Tobago     5621.368472     6619.551419     7899.554209
United States          19530.365570    21806.035940    24072.632130
Uruguay                 5444.619620     5703.408898     6504.339663
Venezuela               9541.474188    10505.259660    13143.950950

                     gdpPercap_1982  gdpPercap_1987  gdpPercap_1992  \
country
Argentina               8997.897412     9139.671389     9308.418710
Bolivia                 3156.510452     2753.691490     2961.699694
Brazil                  7030.835878     7807.095818     6950.283021
Canada                 22898.792140    26626.515030    26342.884260
Chile                   5095.665738     5547.063754     7596.125964
Colombia                4397.575659     4903.219100     5444.648617
Costa Rica              5262.734751     5629.915318     6160.416317
Cuba                    7316.918107     7532.924763     5592.843963
Dominican Republic      2861.092386     2899.842175     3044.214214
Ecuador                 7213.791267     6481.776993     7103.702595
El Salvador             4098.344175     4140.442097     4444.231700
Guatemala               4820.494790     4246.485974     4439.450840
Haiti                   2011.159549     1823.015995     1456.309517
Honduras                3121.760794     3023.096699     3081.694603
Jamaica                 6068.051350     6351.237495     7404.923685
Mexico                  9611.147541     8688.156003     9472.384295
Nicaragua               3470.338156     2955.984375     2170.151724
Panama                  7009.601598     7034.779161     6618.743050
Paraguay                4258.503604     3998.875695     4196.411078
Peru                    6434.501797     6360.943444     4446.380924
Puerto Rico            10330.989150    12281.341910    14641.587110
Trinidad and Tobago     9119.528607     7388.597823     7370.990932
United States          25009.559140    29884.350410    32003.932240
Uruguay                 6920.223051     7452.398969     8137.004775
Venezuela              11152.410110     9883.584648    10733.926310

                     gdpPercap_1997  gdpPercap_2002  gdpPercap_2007
country
Argentina              10967.281950     8797.640716    12779.379640
Bolivia                 3326.143191     3413.262690     3822.137084
Brazil                  7957.980824     8131.212843     9065.800825
Canada                 28954.925890    33328.965070    36319.235010
Chile                  10118.053180    10778.783850    13171.638850
Colombia                6117.361746     5755.259962     7006.580419
Costa Rica              6677.045314     7723.447195     9645.061420
Cuba                    5431.990415     6340.646683     8948.102923
Dominican Republic      3614.101285     4563.808154     6025.374752
Ecuador                 7429.455877     5773.044512     6873.262326
El Salvador             5154.825496     5351.568666     5728.353514
Guatemala               4684.313807     4858.347495     5186.050003
Haiti                   1341.726931     1270.364932     1201.637154
Honduras                3160.454906     3099.728660     3548.330846
Jamaica                 7121.924704     6994.774861     7320.880262
Mexico                  9767.297530    10742.440530    11977.574960
Nicaragua               2253.023004     2474.548819     2749.320965
Panama                  7113.692252     7356.031934     9809.185636
Paraguay                4247.400261     3783.674243     4172.838464
Peru                    5838.347657     5909.020073     7408.905561
Puerto Rico            16999.433300    18855.606180    19328.709010
Trinidad and Tobago     8792.573126    11460.600230    18008.509240
United States          35767.433030    39097.099550    42951.653090
Uruguay                 9230.240708     7727.002004    10611.462990
Venezuela              10165.495180     8605.047831    11415.805690
```

Now want to see what we can do with this data

Some easy things:

In [6]:

```python
data_americas.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 25 entries, Argentina to Venezuela
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   gdpPercap_1952  25 non-null     float64
 1   gdpPercap_1957  25 non-null     float64
 2   gdpPercap_1962  25 non-null     float64
 3   gdpPercap_1967  25 non-null     float64
 4   gdpPercap_1972  25 non-null     float64
 5   gdpPercap_1977  25 non-null     float64
 6   gdpPercap_1982  25 non-null     float64
 7   gdpPercap_1987  25 non-null     float64
 8   gdpPercap_1992  25 non-null     float64
 9   gdpPercap_1997  25 non-null     float64
 10  gdpPercap_2002  25 non-null     float64
 11  gdpPercap_2007  25 non-null     float64
dtypes: float64(12)
memory usage: 2.5+ KB
```

In [7]:

```python
data_americas.describe()
```

Out[7]:

	gdpPercap_1952	gdpPercap_1957	gdpPercap_1962	gdpPercap_1967	gdpPercap_1972	gdpPercap_1977	gdpPercap_1982	gdpPercap_1987	gdpPercap_1992	gdpPercap_1997	gdpPercap_200
count	25.000000	25.000000	25.000000	25.000000	25.000000	25.000000	25.000000	25.000000	25.000000	25.000000	25.00000
mean	4079.062552	4616.043733	4901.541870	5668.253496	6491.334139	7352.007126	7506.737088	7793.400261	8044.934406	8889.300863	9287.67710
std	3001.727522	3312.381083	3421.740569	4160.885560	4754.404329	5355.602518	5530.490471	6665.039509	7047.089191	7874.225145	8895.81778
min	1397.717137	1544.402995	1662.137359	1452.057666	1654.456946	1874.298931	2011.159549	1823.015995	1456.309517	1341.726931	1270.36493
25%	2428.237769	2487.365989	2750.364446	3242.531147	4031.408271	4756.763836	4258.503604	4140.442097	4439.450840	4684.313807	4858.34749
50%	3048.302900	3780.546651	4086.114078	4643.393534	5305.445256	6281.290855	6434.501797	6360.943444	6618.743050	7113.692252	6994.77486
75%	3939.978789	4756.525781	5180.755910	5788.093330	6809.406690	7674.929108	8997.897412	7807.095818	8137.004775	9767.297530	8797.64071
max	13990.482080	14847.127120	16173.145860	19530.365570	21806.035940	24072.632130	25009.559140	29884.350410	32003.932240	35767.433030	39097.09955

`df.info()` gives basic information about a dataframe, with info about each column of data (series)

`df.describe()` gives basic statistics for each column: mean, standard deviation, min/max values, and 25th, 50th, and 75th percentiles in data

Also: `df.columns()` prints column names, `df.T` transposes the data, so

In [8]:

```python
data_americas.T.describe()
```

Out[8]:

country	Argentina	Bolivia	Brazil	Canada	Chile	Colombia	Costa Rica	Cuba	Dominican Republic	Ecuador	...	Mexico	Nicaragua	Panama
count	12.000000	12.000000	12.000000	12.000000	12.000000	12.000000	12.000000	12.000000	12.000000	12.000000	...	12.000000	12.000000	12.000000
mean	8955.553783	2961.228754	5829.316653	22410.746340	6703.289147	4195.342920	5448.610779	6283.258678	2844.856405	5733.624767	...	7724.112675	3424.655770	5754.827172
std	1862.583151	524.748706	2436.453754	8210.112789	3026.194023	1652.931466	2018.532901	1127.781072	1373.906233	1435.152282	...	2770.812253	1044.602174	2139.350722
min	5911.315053	2127.686326	2108.944355	11367.161120	3939.978789	2144.115096	2627.009471	5180.755910	1397.717137	3522.110717	...	3478.125529	2170.151724	2480.380334
25%	7823.006272	2654.716274	3406.544718	15423.062410	4697.346460	2632.135156	3986.530132	5547.901689	1660.033770	4455.834181	...	5461.452758	2680.627929	4199.891888
50%	9068.784401	2971.015516	6805.200838	22494.837600	5300.339375	4106.691765	5446.325034	5891.221187	2771.540643	6127.410752	...	8181.542556	3284.889947	5991.496356
75%	9602.035579	3347.923066	7844.817069	27208.617745	8226.607768	5522.301453	6289.573566	6614.600751	3186.685982	6930.872393	...	9650.185038	3886.621688	7054.507434
max	12779.379640	3822.137084	9065.800825	36319.235010	13171.638850	7006.580419	9645.061420	8948.102923	6025.374752	7429.455877	...	11977.574960	5486.371089	9809.185636

8 rows × 25 columns

gives same statistics, but now by country instead of by year.

Can grab the first or last few rows using `.head()` and `.tail()`

In [9]:

```python
data_americas.head()
```

Out[9]:

country	gdpPercap_1952	gdpPercap_1957	gdpPercap_1962	gdpPercap_1967	gdpPercap_1972	gdpPercap_1977	gdpPercap_1982	gdpPercap_1987	gdpPercap_1992	gdpPercap_1997	gdpPercap_
Argentina	5911.315053	6856.856212	7133.166023	8052.953021	9443.038526	10079.026740	8997.897412	9139.671389	9308.418710	10967.281950	8797.6
Bolivia	2677.326347	2127.686326	2180.972546	2586.886053	2980.331339	3548.097832	3156.510452	2753.691490	2961.699694	3326.143191	3413.26
Brazil	2108.944355	2487.365989	3336.585802	3429.864357	4985.711467	6660.118654	7030.835878	7807.095818	6950.283021	7957.980824	8131.2
Canada	11367.161120	12489.950060	13462.485550	16076.588030	18970.570860	22090.883060	22898.792140	26626.515030	26342.884260	28954.925890	33328.9
Chile	3939.978789	4315.622723	4519.094331	5106.654313	5494.024437	4756.763836	5095.665738	5547.063754	7596.125964	10118.053180	10778.7

In [10]:

```python
data_americas.tail(n=3)
```

Out[10]:

country	gdpPercap_1952	gdpPercap_1957	gdpPercap_1962	gdpPercap_1967	gdpPercap_1972	gdpPercap_1977	gdpPercap_1982	gdpPercap_1987	gdpPercap_1992	gdpPercap_1997	gdpPercap_
United States	13990.482080	14847.127120	16173.145860	19530.365570	21806.035940	24072.632130	25009.559140	29884.350410	32003.932240	35767.433030	39097.0
Uruguay	5716.766744	6150.772969	5603.357717	5444.619620	5703.408898	6504.339663	6920.223051	7452.398969	8137.004775	9230.240708	7727.0
Venezuela	7689.799761	9802.466526	8422.974165	9541.474188	10505.259660	13143.950950	11152.410110	9883.584648	10733.926310	10165.495180	8605.0

Can select data from a dataframe by labels, or by indices:

In [ ]:

```python
print(data_americas.loc["Canada"])
```

```
gdpPercap_1952    11367.16112
gdpPercap_1957    12489.95006
gdpPercap_1962    13462.48555
gdpPercap_1967    16076.58803
gdpPercap_1972    18970.57086
gdpPercap_1977    22090.88306
gdpPercap_1982    22898.79214
gdpPercap_1987    26626.51503
gdpPercap_1992    26342.88426
gdpPercap_1997    28954.92589
gdpPercap_2002    33328.96507
gdpPercap_2007    36319.23501
Name: Canada, dtype: float64
```

In [13]:

```python
print(data_americas.loc["Canada", "gdpPercap_2007"])
```

```
36319.23501
```

In [14]:

```python
print(data_americas.iloc[3,11])
```

```
36319.23501
```

Using indices is a bit inconvenient, requires you to know correspondence between data and numerical indices, so tend to use labels

Can also do slicing:

In [15]:

```python
print(data_americas.loc[:,"gdpPercap_2007"])
```

```
country
Argentina              12779.379640
Bolivia                 3822.137084
Brazil                  9065.800825
Canada                 36319.235010
Chile                  13171.638850
Colombia                7006.580419
Costa Rica              9645.061420
Cuba                    8948.102923
Dominican Republic      6025.374752
Ecuador                 6873.262326
El Salvador             5728.353514
Guatemala               5186.050003
Haiti                   1201.637154
Honduras                3548.330846
Jamaica                 7320.880262
Mexico                 11977.574960
Nicaragua               2749.320965
Panama                  9809.185636
Paraguay                4172.838464
Peru                    7408.905561
Puerto Rico            19328.709010
Trinidad and Tobago    18008.509240
United States          42951.653090
Uruguay                10611.462990
Venezuela              11415.805690
Name: gdpPercap_2007, dtype: float64
```

In [16]:

```python
print(data_americas.loc["Canada":"Cuba", "gdpPercap_1987":"gdpPercap_2002"])
```

```
            gdpPercap_1987  gdpPercap_1992  gdpPercap_1997  gdpPercap_2002
country
Canada        26626.515030    26342.884260    28954.925890    33328.965070
Chile          5547.063754     7596.125964    10118.053180    10778.783850
Colombia       4903.219100     5444.648617     6117.361746     5755.259962
Costa Rica     5629.915318     6160.416317     6677.045314     7723.447195
Cuba           7532.924763     5592.843963     5431.990415     6340.646683
```

Can run statistics on these slices

In [17]:

```python
subset = data_americas.loc["Canada":"Cuba", "gdpPercap_1987":"gdpPercap_2002"]
print(subset.max())
```

```
gdpPercap_1987    26626.51503
gdpPercap_1992    26342.88426
gdpPercap_1997    28954.92589
gdpPercap_2002    33328.96507
dtype: float64
```

In [18]:

```python
print(subset.min())
```

```
gdpPercap_1987    4903.219100
gdpPercap_1992    5444.648617
gdpPercap_1997    5431.990415
gdpPercap_2002    5755.259962
dtype: float64
```

Define "masks" based on comparisons

In [19]:

```python
mask = subset > 6000
print(mask)
```

```
            gdpPercap_1987  gdpPercap_1992  gdpPercap_1997  gdpPercap_2002
country
Canada                True            True            True            True
Chile                False            True            True            True
Colombia             False           False            True           False
Costa Rica           False            True            True            True
Cuba                  True           False           False            True
```

Shows True/False values for this comparison

In [20]:

```python
print(subset[mask])
```

```
            gdpPercap_1987  gdpPercap_1992  gdpPercap_1997  gdpPercap_2002
country
Canada        26626.515030    26342.884260    28954.925890    33328.965070
Chile                  NaN     7596.125964    10118.053180    10778.783850
Colombia               NaN             NaN     6117.361746             NaN
Costa Rica             NaN     6160.416317     6677.045314     7723.447195
Cuba           7532.924763             NaN             NaN     6340.646683
```

Gives values where comparisons are true and Not-a-Number (NaN) when they are false

This is useful to do statistics over masked data

```
In [21]: print(subset[mask].describe())
```

```
       gdpPercap_1987  gdpPercap_1992  gdpPercap_1997  gdpPercap_2002
count        2.000000        3.000000        4.000000        4.000000
mean     17079.719896    13366.475514    12966.846533    14542.960699
std      13501.207155    11260.803857    10804.495150    12660.525361
min       7532.924763     6160.416317     6117.361746     6340.646683
25%      12306.322330     6878.271141     6537.124422     7377.747067
50%      17079.719896     7596.125964     8397.549247     9251.115523
75%      21853.117463    16969.505112    14827.271357    16416.329155
max      26626.515030    26342.884260    28954.925890    33328.965070
```